

Kmeans_{report}*Redacted*

Part 1: Implementing K-means from Scratch

- `initialize_centroids(X, k, random_state=42)`
- `assign_clusters(X, centroids)`
- `update_centroids(X, labels, k)`
- `kmeans_from_scratch(X, k, max_iter=100, tol=1e-4, random_state=42)`

The implementation uses Euclidean distance, checks convergence based on centroid movement, and returns the final labels, centroids, and iteration count.

Key code block

```
def initialize_centroids(X, k, random_state=42):
    rng = np.random.RandomState(random_state)
    idx = rng.choice(X.shape[0], size=k, replace=False)
    return X[idx].copy()

def assign_clusters(X, centroids):
    distances = np.linalg.norm(X[:, None, :] - centroids[None, :, :], axis=2)
    return np.argmin(distances, axis=1)

def update_centroids(X, labels, k):
    centroids = []
    for i in range(k):
        cluster_points = X[labels == i]
        if len(cluster_points) == 0:
            centroids.append(np.zeros(X.shape[1]))
        else:
            centroids.append(cluster_points.mean(axis=0))
    return np.array(centroids)

def kmeans_from_scratch(X, k, max_iter=100, tol=1e-4, random_state=42):
    centroids = initialize_centroids(X, k, random_state=random_state)
    for it in range(max_iter):
        labels = assign_clusters(X, centroids)
        new_centroids = update_centroids(X, labels, k)
        shift = np.linalg.norm(new_centroids - centroids)
        centroids = new_centroids
        if shift < tol:
            return labels, centroids, it + 1
    return labels, centroids, max_iter
```

Part 2: Using scikit-learn and Visualization

A synthetic dataset with 3 centers, 300 samples, and 2 features was generated using `make_blobs`. The data were standardized by `StandardScaler`, then clustered using both the scratch implementation and scikit-learn's `KMeans` under the same setting.

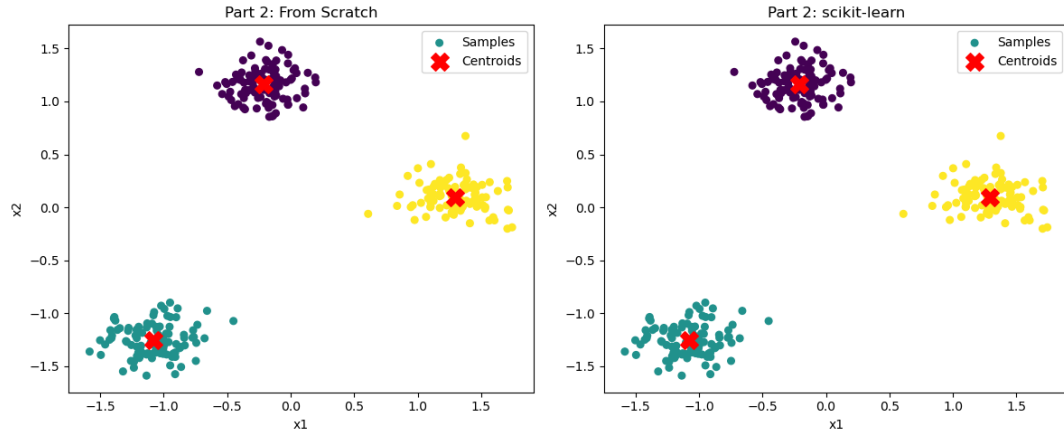


Figure 1: Comparison of clustering results: left is the scratch implementation, right is scikit-learn `KMeans`. Data points are colored by cluster assignment and centroids are marked by red X symbols.

Analysis Both methods optimize the same K-means objective, so their clustering results are expected to be almost identical up to label permutation and centroid ordering. After standardization and using the same random seed, the two subplots show nearly the same partition structure. Any small difference is usually caused by initialization details or numerical stopping conditions rather than a fundamental difference in the algorithm.

Part 3: Determining the Optimal Number of Clusters

Another synthetic dataset with 5 centers, 500 samples, and 2 features was generated and standardized. I applied both the elbow method and silhouette analysis to determine the optimal number of clusters.

Elbow method For each $k = 1, \dots, 10$, I ran K-means and recorded the inertia.

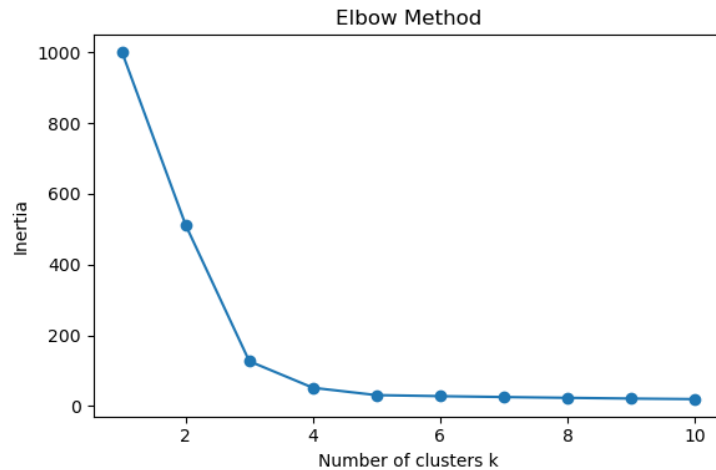


Figure 2: Elbow curve: inertia versus number of clusters k .

Silhouette scores For each $k = 2, \dots, 10$, I computed the average silhouette score.

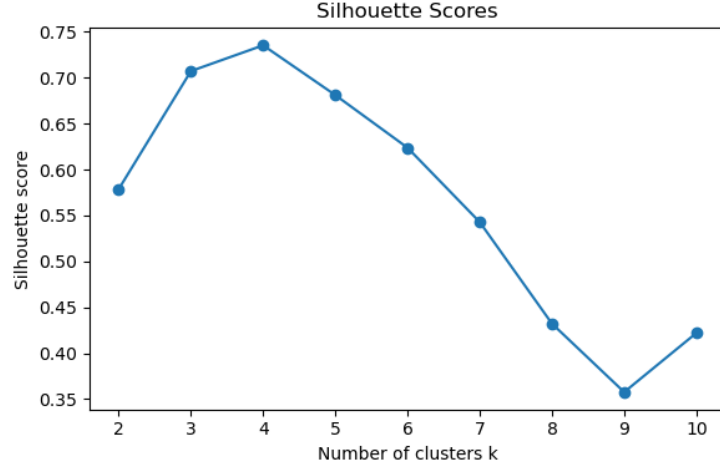


Figure 3: Average silhouette scores for different values of k .

Silhouette plots To further inspect cluster quality, I generated silhouette plots for $k = 3$, $k = 5$, and $k = 7$.

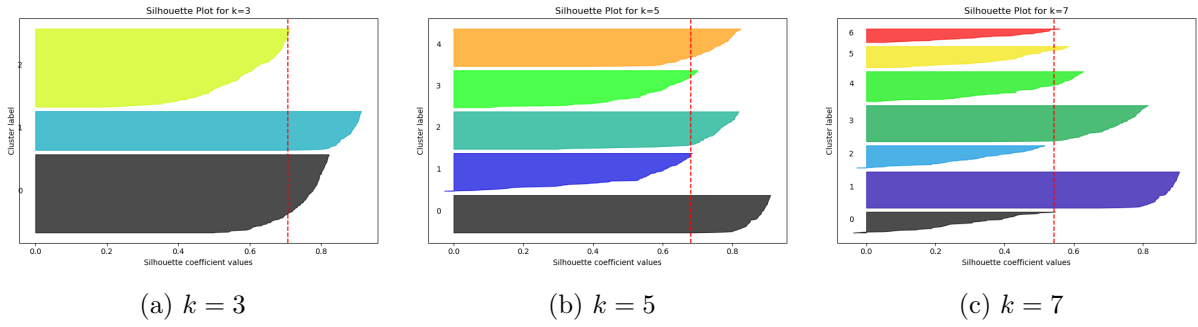


Figure 4: Silhouette plots for three candidate values of k .

Final clustering result Based on the above two methods, I selected the optimal number of clusters as

$$k = 5.$$

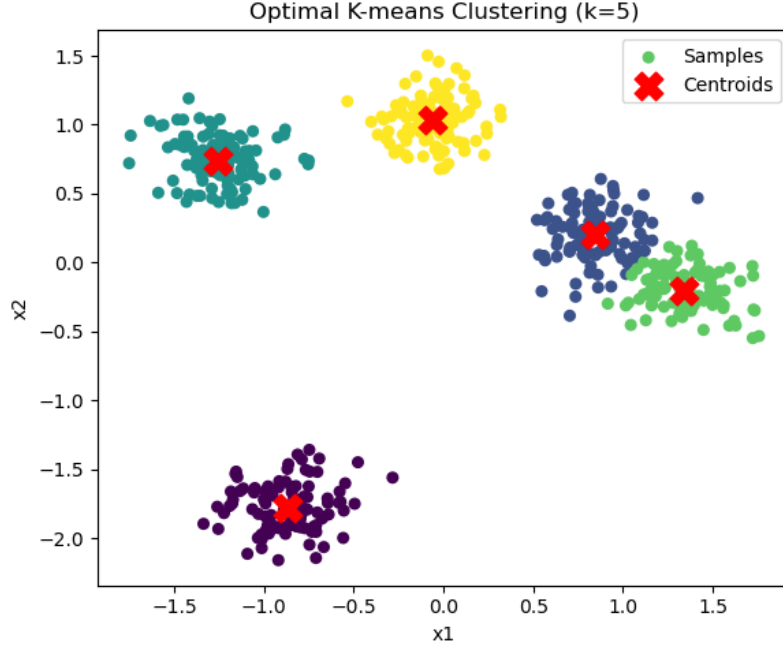


Figure 5: Final clustering result using the selected optimal number of clusters, with centroids marked by red X symbols.

Analysis The elbow plot shows a clear turning point around $k = 5$, after which the inertia still decreases but at a much slower rate. The silhouette score is also expected to be highest or near-highest at $k = 5$, indicating better separation and compactness than most other choices. Looking at the silhouette plots, $k = 3$ tends to merge several true groups together, while $k = 7$ tends to split natural clusters into smaller pieces. In contrast, $k = 5$ gives a more balanced structure, cleaner separation, and a clustering result that matches the data generation setting best. Therefore, $k = 5$ is the most reasonable choice for this dataset.